



DATA STRUCTURE AND ALGORITHM

Programming Practice Session

10/12/2025



Revision

Revision



1. A person's daily movement schedule is stored in a singly linked list in ascending order of time. Each node is defined as follows:

```
typedef struct aPlace{  
    int hour, min;      // the time of the visit (hour and minute)  
    char locName[255];  // name of the location  
    struct aPlace* next; // pointer to the next node  
} PLACE;
```

Suppose that, for COVID contact tracing purposes, the Preventive Health Department announces that anyone who has passed through a location named reportLocName after the time reportHour and reportMin (on the same day) must file a medical report.

The program (including function below) returns 1 if the individual needs to file a medical report, and 0 otherwise. Here, visitedPlaces is the head pointer of the linked list containing the locations the person has visited along with the time of each visit.

```
int needMedicalReport(PLACE* visitedPlaces, char* reportLocName, int  
reportHour, int reportMin);
```



2. Given two standard queues, design a sorting algorithm that uses only these two queues (without any additional auxiliary memory) to sort a list of n integer elements. The only allowed operations are enqueue and dequeue.

Input:

7

5 1 4 2 6 3 1

Output:

1 1 2 3 4 5 6



3. A double-ended queue (Deque) is a data structure that allows insertion and deletion of elements at both ends.

Write a program to implement a Deque and process the following queries:

- insertFront x – Insert element x at the front of the deque
- insertBack x – Insert element x at the back of the deque
- eraseFront – Remove the element at the front. If the deque is empty, ignore this operation
- eraseBack – Remove the element at the back. If the deque is empty, ignore this operation
- front – Print the value of the front element. If the deque is empty, print "EMPTY"
- back – Print the value of the back element. If the deque is empty, print "EMPTY"
- isEmpty – Print "YES" if the deque is empty, otherwise print "NO"



4. A **Red-Black Tree (RBT)** is a Binary Search Tree (BST) in which each node stores an additional color attribute (red or black), and the tree must satisfy the following rules:

1. Every node is either red or black.
2. The root of the tree is always black.
3. All leaf nodes (NIL nodes) are black.
4. If a node is red, then both of its children must be black.
5. For every node, all simple paths from that node to its NIL descendants contain the same number of black nodes.

A Red-Black Tree supports searching, inserting, and deleting elements in **$O(\log n)$** time. Implement a Red-Black Tree that stores integers and process the following queries:

- insert x – Insert value x into the RBT. If x already exists, ignore the operation.
- delete x – Delete value x from the tree if it exists.
- find x – Search for value x; print "YES" if it exists, otherwise "NO".
- printInorder – Print all values in the tree in increasing order.
- printColor x – Print the color of the node containing value x (RED or BLACK). If the value does not exist, print "Not found".



5. Write a C++ program that uses a Doubly Linked List to manage a list of customers waiting to buy train tickets at a station. Each customer record contains: ID number (string up to 10 characters, unique), Customer name, Destination station, Ticket price. Your program must provide a menu with the following features:

- Load the waiting list from file. Each line in the file contains: ID;Name;Station;Price.
- Add a new customer to the waiting queue. If already exists, do not create a new node.
- Sell a ticket to a customer who registered first.
- Display the customer list
- Remove a customer from the list (the customer no longer wants to buy a ticket)
- Statistics of ticket sales
- Save the list to file
- Display the list of stations currently being waited for
- Display the list of stations being waited for and the number of customers waiting at each station